Collaboration Hub

Technologies

Frontend (Angular SPA)

- **Angular & TypeScript:** Core framework and language.
- **RxJS:** State management and asynchronous WebSocket streams.
- **HTML5 & SCSS:** UI templating and styling.
- **Nginx:** Lightweight web server for production serving.

Backend (Spring Boot API)

- **Java 17+ & Spring Boot:** Core backend logic and REST API.
- **Spring WebSockets & STOMP:** Real-time data synchronization.
- **Spring Security & JWT:** Authentication and stateless authorization.
- **Spring Data MongoDB:** Object-document mapping.
- **Gradle & Lombok:** Build automation and boilerplate reduction.

Database

- **MongoDB:** NoSQL document database for persisting sprints and users.

DevOps & Infrastructure

- **Docker & Docker Compose:** Containerization of all services.
- **GitHub Actions:** CI/CD pipelines (automated build & deployment).
- **Google Cloud Platform (GCP):** Production hosting environment (VM).

Workflow & Engineering Practices

- **Prototyping & Design:** Initial UI/UX concepts and component layouts were developed using **Figma** before any frontend implementation.
- **Backlog Management:** The Product and Sprint backlogs were continuously managed and tracked using **GitHub Projects**, ensuring full visibility.
- **Strict Branching Strategy:** Collaborative Git workflow enforced via Pull Requests (PRs). Code reviews were mandatory, and *only* accepted PRs were allowed to be merged into the `main` branch.
- **CI/CD Pipeline:** Fully automated workflow configured using **GitHub Actions** (`deploy.yaml`, `main.yml`).
- **Seamless Deployment:** Upon successful testing and building, the pipeline automatically containerizes the application via Docker and deploys it live to our Google Cloud Platform (GCP) server.
- **Repository Documentation:** `README.md` provided, including clear, step-by-step instructions for database configuration, local environment setup, running tests, and deployment.

US 1.1 – Define Sprint Goal

- Implemented using Angular’s ReactiveFormsModule (goalForm) for robust input handling.
- The goal is bound to a sprintGoal component state variable.
- Updates are managed via an updateGoal() method which syncs the state with the UI.

US 1.2 – User Stories

- Handled via an Angular `storyForm` to capture titles and optional descriptions.
- Stories are stored in an in-memory `UserStory[]` array.
- New stories are dynamically appended with a default status of "To Do".

US 1.3 – Progress Status

- Implemented using changeStatus() and changeTaskStatus() methods in the frontend component.
- Allows the developer to modify the status (To Do, In Progress, Done) directly on the specific story or task object.
- Angular's data binding automatically reflects these state changes on the visual board.

US 1.4 – Progress Overview

- Calculated dynamically via a progressPercentage getter in the TypeScript component.
- Computes the completion ratio by dividing stories with a "Done" status by the total number of stories.
- Provides a real-time metric (0-100%) without requiring a page refresh.

US 1.5 – Browser Compatibility

- Built as a Single Page Application (SPA) using standard web technologies (HTML5, SCSS, TypeScript).
- Compiled and polyfilled by the Angular CLI to ensure native execution on modern browsers.
- Tested to run on Google Chrome v99+ without relying on deprecated web APIs.

US 1.6 – Usability

- Clean, intuitive UI logic requiring no external documentation.
- Clear action buttons (e.g., "Add Story", "Complete Sprint") and straightforward form structures.
- Immediate feedback mechanisms, such as preventing edits if a user is not logged in.

US 2.1 – Create Sprint Session

- Frontend triggers backend SprintController to generate a random 6-character, uppercase UUID snippet.
- A new SprintSession document is instantiated and saved to the MongoDB database.
- The frontend automatically connects to the session's specific WebSocket topic upon creation.

US 2.2 – Join Sprint Session

- Users input a 6-character session code which is normalized and sent to the backend.
- The backend retrieves the session from the SprintRepository and adds the user to the active participant list.
- Frontend sets up an active WebSocket and a heartbeat interval (presenceSubscription) to maintain active status.

US 2.3 – Shared Progress

- Real-time synchronization implemented using WebSockets (STOMP protocol) and Spring's SimpMessagingTemplate.
- The backend broadcasts updates (/topic/sprints/{sessionId}) whenever the session state changes.
- The frontend subscribes to *sessionUpdates$* and intelligently merges incoming stories and user presence data without disrupting local input.

US 2.4 – Sprint Goal Update

- Modifying the goal form triggers a PUT request to update the session in MongoDB.
- The backend updates the session and fires a broadcast message to all connected clients.
- The frontend receives the update and checks if the local form is "pristine" before safely overwriting the visual goal state to prevent typing conflicts.

US 3.1 – Sprint History

- Backend exposes a GET /api/sprints/history endpoint filtering for completed=true sessions linked to the user's ID.
- The completeSprint() method allows users to mark an active session as finished, triggering a database update.
- The frontend allows authenticated users to toggle the history view (showHistory), dynamically loading past iterations.

US 3.2 – Login and Registration

- Secured via Spring Security and JSON Web Tokens (JWT) on the Java backend.
- The backend SprintController validates the AuthenticatedUser principal before allowing any state modifications (PUT/POST/DELETE).
- The frontend AuthService manages token storage, enforcing view-only modes for unauthenticated users (this.isLoggedIn).

US 3.3 – Delete Confirmation

- Implemented defensively on the frontend using native browser confirm() dialogs.
- Used during critical actions (e.g., completing a sprint) to prevent accidental data loss.
- Ensures explicit user consent before moving active sprints into the immutable history state.

UC 3.4 – Deployment

- Entire stack is containerized using Docker, managed via a docker-compose.yml file (MongoDB, Backend, Frontend).
- Automated CI/CD pipeline set up using GitHub Actions (deploy.yaml).
- Pushes to the Deploy branch automatically SSH into a Google Cloud Platform (GCP) VM, pull the latest code, build the images, and restart the containers.